Memorandum

To: Charlie Refvem, Department of Mechanical Engineering, Cal Poly SLO
crefvem@calpoly.edu

From: Lucas Nathan
lrnathan@calpoly.edu

Vandan Patel
vpatel43@calpoly.edu

Oliver Lenoir
olenoir@calpoly

Date: 22 February 2026

RE: **Lab 0x04 - UI Design and Automated Data Collection**

Overview of Program Flow

The program for Lab 0x04 was designed using a cooperative multitasking structure on the Nucleo microcontroller together with an automated Python data collection script running on a laptop. The purpose of this design was to allow repeatable step response testing while maintaining reliable real time motor control and enabling systematic controller tuning.

On the microcontroller, the system is organized into three tasks. Two tasks are responsible for closed loop motor control, one for each motor. The third task manages the user interface and serial communication. These tasks are scheduled using a priority based cooperative scheduler. The motor control tasks run at a fixed period of 50 ms and perform encoder updates, velocity calculations, PI control, and motor actuation.

Each motor task is implemented as a finite state machine with three states: initialization, waiting for a trigger signal, and closed loop control with data collection. When a step response is triggered, both motor tasks begin recording timestamped velocity data into shared queues. Data collection continues until the buffers are full, at which point the motors are stopped and the tasks return to the waiting state. The user interface task then prints the collected data in CSV format over the serial connection.

The laptop Python script automates the tuning process by sending gain values and velocity setpoints directly to the microcontroller through the serial port. Instead of manually entering commands, the script sends the gain command, transmits the selected $K_p$and $K_i$values, sends the setpoint command, and then triggers a step response. The script waits for the CSV header, collects the incoming data, and automatically saves both a CSV file and a corresponding plot for each test.

This automated structure made it possible to compare controller performance across multiple tuning conditions. For example, Figure B-1 shows the response using proportional control only. The system reaches the commanded velocity but exhibits a small steady state error due to the

absence of integral action. In Figure B-2, the addition of a small integral gain improves rise time and reduces steady state error, although some oscillation appears during the transient response. In Figure B-3, a larger integral gain further reduces steady state error but increases oscillation and makes the response more aggressive.

By separating motor control on the microcontroller from automated testing and plotting on the laptop, the system allows quick iteration of gain values and clear comparison of performance. The plots demonstrate how changes in proportional and integral gains directly affect rise time, steady state error, and stability. This structure made the tuning process more efficient and allowed clear evaluation of controller performance.

The best performance was achieved with Kp=1.5 and Ki=0.05 (Figure B-2). The addition of small integral gain over proportional-only control significantly reduced steady state error and improved the rise time. Increasing Ki further (Figure B-3) introduced more oscillation, making the response less stable.

Appendix A. Diagrams

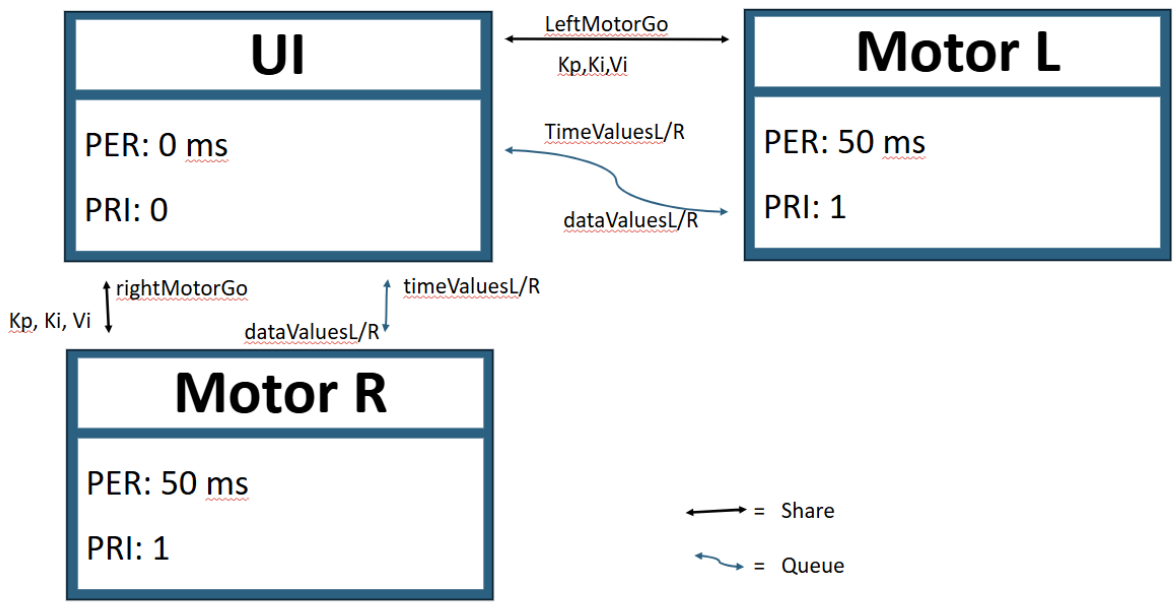

Figure A-1. Task Diagram. Shows how the three tasks communicate with each other and the scheduler.

The UI period and priority were chosen with the intent of the UI running at lowest priority, whenever the task manager had downtime.

Both motors having the same priority is to ensure symmetric control behavior

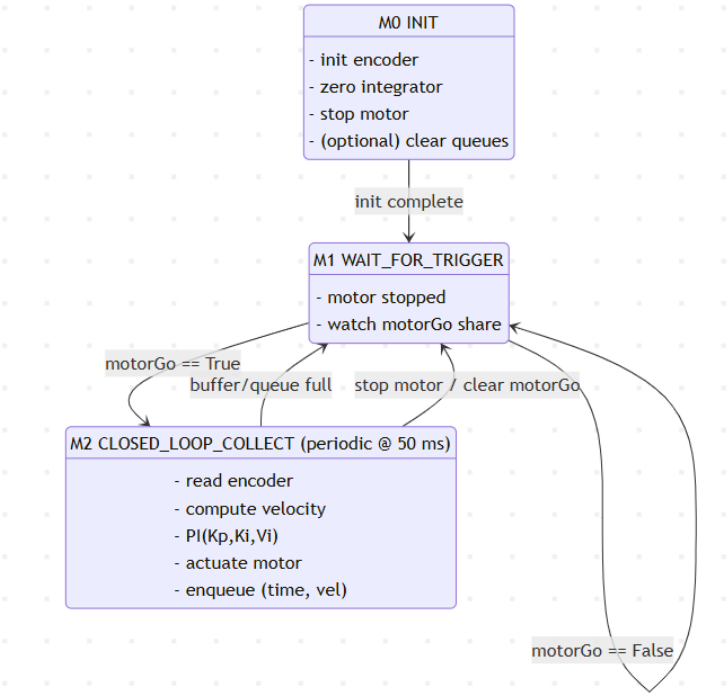

Figure A-2. Motor Task Finite State Machines (same for both left and right).

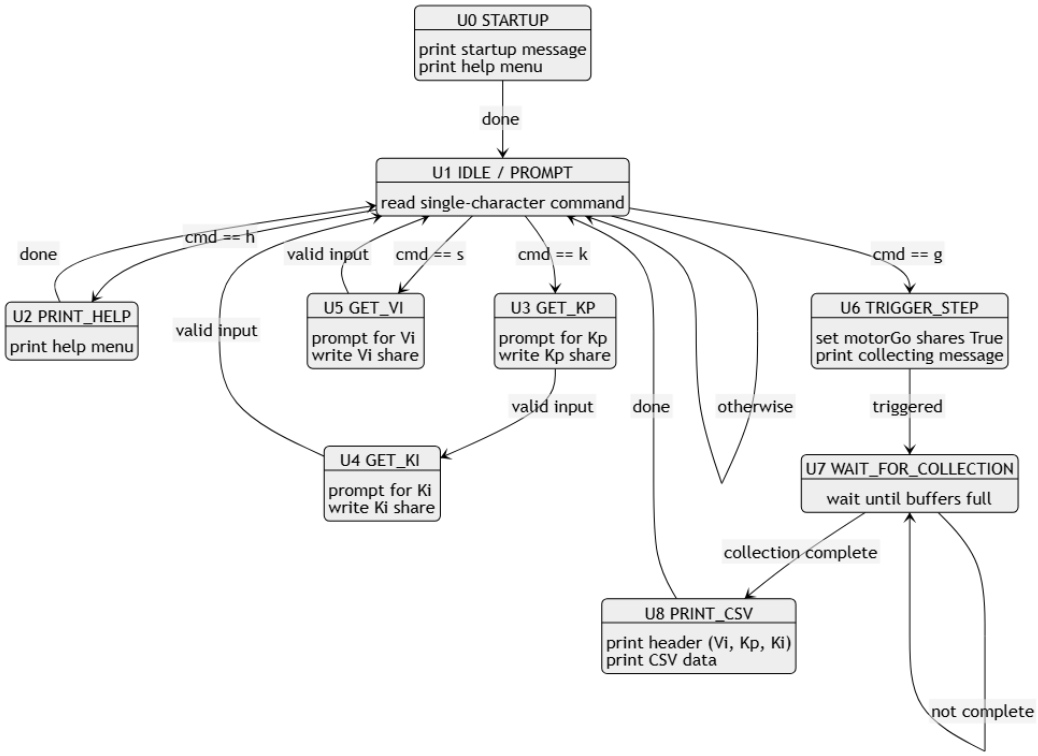

Figure A-3. User Interface Finite State Machine.

Appendix B. Plots

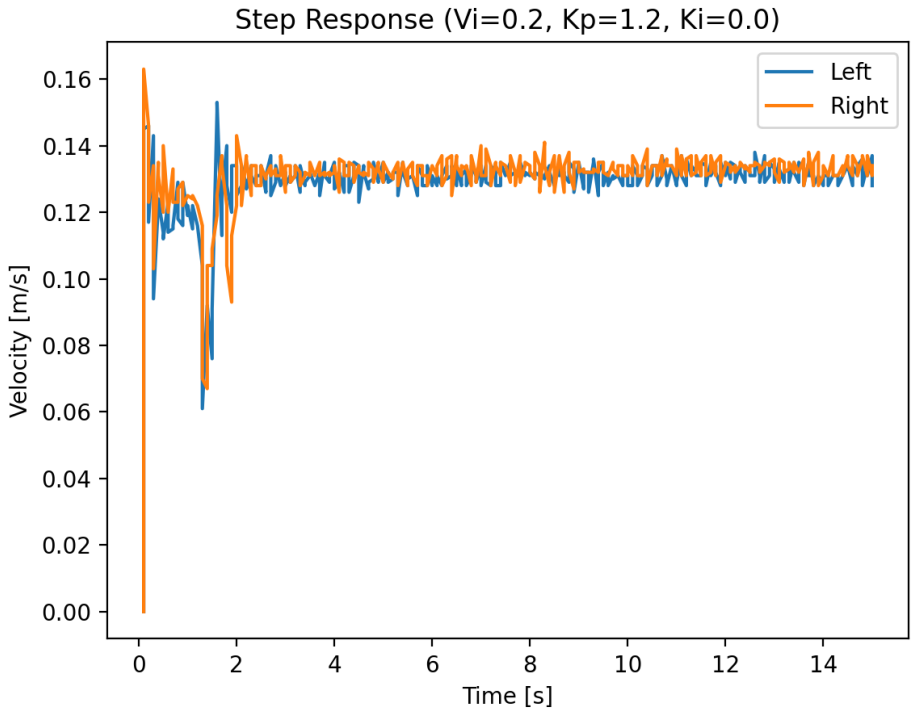

Figure B-1. Step Response for Vi = 0.2 m/s, Kp = 1.2, Ki = 0. This plot shows the step response using proportional control only. The system reaches the commanded velocity with moderate rise time and minimal overshoot. A small steady state error is present due to the absence of integral control. Both left and right motors exhibit similar behavior with slight measurement noise.

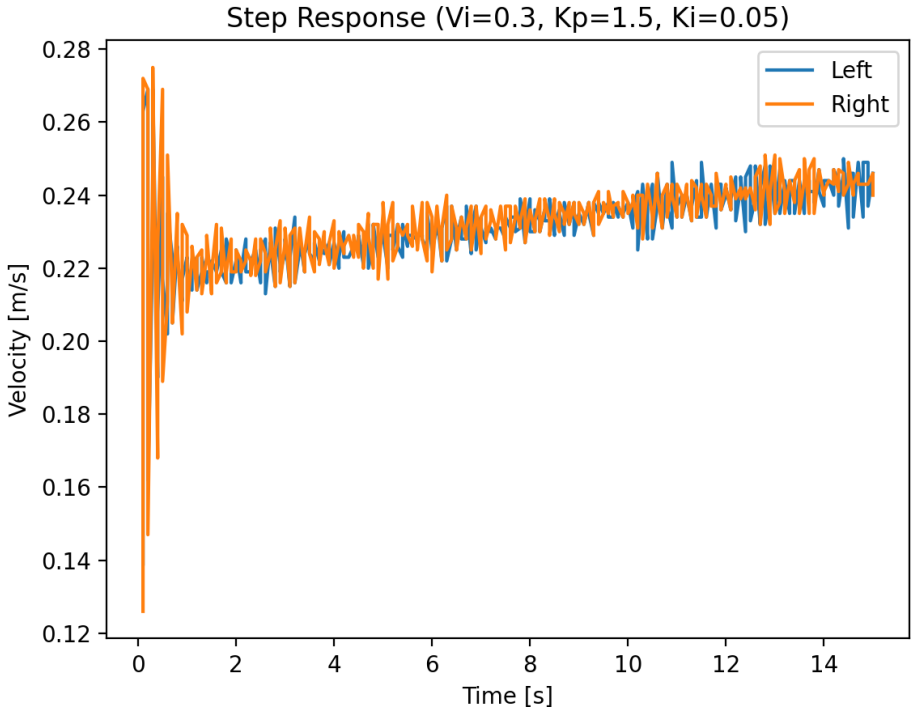

Figure B-2. Step Response for Vi = 0.3 m/s, Kp = 1.5, Ki = 0.05. This response includes a small integral gain. The rise time improves compared to the previous case, and steady state error is reduced. However, slight oscillation and increased noise are observed during the transient response. Both motors track the setpoint closely after settling.

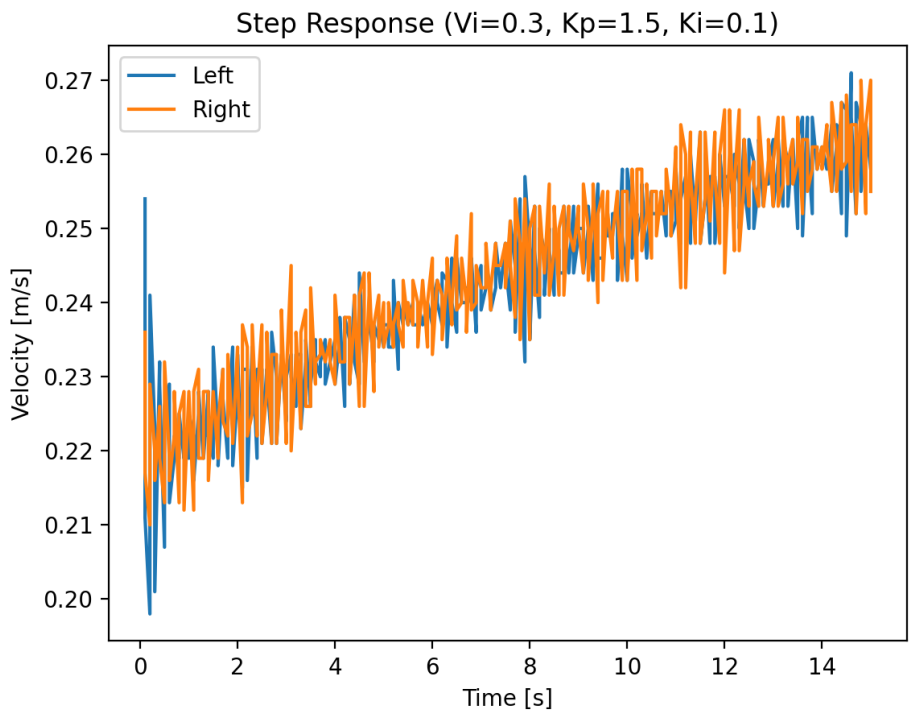

Figure B-3. Step Response for Vi = 0.3 m/s, Kp = 1.5, Ki = 0.1 . With a larger integral gain, the system further reduces steady state error but shows increased oscillation during the transient period. The response becomes more aggressive and slightly less stable compared to the previous tuning. After settling, both motors maintain the desired velocity with small fluctuations.

Appendix C. Shared Variables

Table C-1. Shared Variables. This table lists all shared variables and queues used in the multitasking system. It identifies the data type stored in each variable and explains how each share or queue supports communication between the user interface and motor control tasks during step response testing and data collection.

Variable Name	Type	Data Stored	Purpose in Task Structure
leftMotorGo	Share (Boolean)	True or False flag	Signals the left motor task to begin a step response and start data collection.
rightMotorGo	Share (Boolean)	True or False flag	Signals the right motor task to begin a step response and start data collection.
Kp	Share (Float)	Proportional gain value	Allows the user interface task to update the proportional gain used by both motor control tasks.
Ki	Share (Float)	Integral gain value	Allows the user interface task to update the integral gain used by both motor control tasks.
Vi	Share (Float)	Velocity setpoint in m/s	Stores the desired velocity setpoint that the motor tasks attempt to track.
dataValuesL	Queue (Float)	Left motor velocity values	Stores collected left motor velocity data during a step response test.
timeValuesL	Queue (Unsigned Int)	Left motor timestamps in microseconds	Stores timestamps corresponding to left motor velocity measurements.
dataValuesR	Queue (Float)	Right motor velocity values	Stores collected right motor velocity data during a step response test.
timeValuesR	Queue (Unsigned Int)	Right motor timestamps in microseconds	Stores timestamps corresponding to right motor velocity measurements.

Appendix D. Task Profile

Table D-1. Task Profiler data. Lists the three tasks, and their respective data from the task profiler.

Task:	UI	Motor L	Motor R
Period [ms]	0	50	50
Priority	0	1	1
Runs	0	2965	2965
Avg Run Time [ms]	0	.120	.128
Max Run Time [ms]	0	.162	.148
Avg Late time [ms]	0	.288	.304
Max Late Time [ms]	0	.591	.598

The profiler data shows that the UI task does not run. This is likely due to how often each motor task ran late(average late times of .288 and .305 ms) However, this data was all sampled during the triggered step response, so no UI inputs were needed/processed. The motor priorities of 50 ms mean an updating frequency of 20hz, which is fast enough to respond smoothly, but slow enough to minimize unnecessary CPU load. Additionally, at higher sample rates, the velocity estimate from the encoder can become jittery.

Appendix E. Functionality Demonstration

```
UI STARTED
User Task object instantiated
+-------------------------------------------------------------------------------+
| ME 405 Romi Tuning Interface Help Menu                                        |
+---+---------------------------------------------------------------------------+
| h | Print help menu                                                           |
| k | Enter new gain values                                                     |
| s | Choose a new setpoint                                                     |
| g | Trigger step response and print results                                   |
+---+---------------------------------------------------------------------------+

>:
```

Figure E-1. UI Interface Demonstration

```
>: k
Enter proportional gain, Kp:
>: 0.2
Kp set to 0.2
Enter integral gain, Ki:
>: 0.2
Ki set to 0.2
```

Figure E-2. Gain Input Demonstration

```
>: g
Step response triggered...
Starting data collection...
Data collection complete...
Printing data...
--------------------
Setpoint: 0.0 [m/s]
Kp: 0.0 [%s/m]
Ki: 0.0 [%/m]
--------------------
Time Left [s], Left Velocity [m/s], Time Right [s], Right Velocity [m/s]
0.1,0.000,0.1,0.000,
0.1,0.000,0.1,0.000,
0.2,0.000,0.2,0.000,
0.2,0.000,0.2,0.000,
0.3,0.000,0.3,0.000,
0.3,0.000,0.3,0.000,
0.4,0.000,0.4,0.000,
0.4,0.000,0.4,0.000,
0.5,0.000,0.5,0.000,
0.5,0.000,0.5,0.000,
0.6,0.000,0.6,0.000,
0.6,0.000,0.6,0.000,
0.7,0.000,0.7,0.000,
0.7,0.000,0.7,0.000,
0.8,0.000,0.8,0.000,
```

Figure E-3. Step Response Trigger and Data Printing Demonstration

```
================================
Saved: D:\DESKTOP\school 25-26\winter 26\me 405\lab\lab 4\02_22_18_40_50.csv
Saved: D:\DESKTOP\school 25-26\winter 26\me 405\lab\lab 4\02_22_18_40_50.png

================================
Running: Kp=1.5, Ki=0.05, Vi=0.3
================================
Saved: D:\DESKTOP\school 25-26\winter 26\me 405\lab\lab 4\02_22_18_41_09.csv
Saved: D:\DESKTOP\school 25-26\winter 26\me 405\lab\lab 4\02_22_18_41_09.png

================================
Running: Kp=1.5, Ki=0.1, Vi=0.3
================================
Saved: D:\DESKTOP\school 25-26\winter 26\me 405\lab\lab 4\02_22_18_41_28.csv
Saved: D:\DESKTOP\school 25-26\winter 26\me 405\lab\lab 4\02_22_18_41_28.png
PS D:\DESKTOP\school 25-26\winter 26\me 405\lab\lab 4>
```

Figure E-4. Data Files and Plots Saved from PC Script

Appendix F. Source Code

main.py

This is the main entry point of the project. It initializes hardware components, creates system tasks, and starts the cooperative task scheduler. It serves as the central controller that brings the entire embedded system together and begins program execution.

cotask.py

This file implements the cooperative multitasking scheduler. It defines task behavior, manages task priorities and states, and runs the scheduling loop. It allows multiple tasks to operate concurrently without preemption.

task_share.py

This file manages shared variables and queues used for communication between tasks. It ensures data can be safely passed between different parts of the system without corruption during multitasking.

task_motor.py

This file contains the motor control task. It likely implements the control algorithm (such as PID control), reads encoder feedback, and determines the appropriate motor command. It manages the overall closed-loop motor behavior.

motor_driver.py

This file provides the low-level interface to the motor hardware. It sets PWM signals, controls motor direction pins, and abstracts the hardware details so higher-level code can control the motor more easily.

encoder.py

This file handles reading the quadrature encoder signals. It tracks motor position and possibly speed, providing feedback data that is used for closed-loop motor control.

task_user.py

This file manages user interaction. It likely handles serial communication, processes user commands, and displays system information or test results. It acts as the interface between the user and the embedded system.

PC code.py

This file runs on a computer and communicates with the embedded system, typically through serial communication. It may send control commands, receive system data, log performance results, and generate plots for analysis.